Hash Table

22 March 2026
23:41

Hash Table

It is not array where the data is stored linear format.
The elements are stored in **KEY -VALUE PAIRS**
It can store multiple data types (bool, string, int)
No Duplicate elements can be stored and search speed is fast
$a = @{ } - empty hash Table

To define a hash table

$a = @{
Name = "John"
Age = 40
Location = "Singapore"
}

```
PS C:\Users\lenovo> $a = @{
>> Name = "John"
>> Age = 40
>> Location = "Singapore"
>> }
PS C:\Users\lenovo> $a

Name                           Value
----                           -----
Name                           John
Age                            40
Location                       Singapore
```

To get the type : **$a.gettype()**

```
PS C:\Users\lenovo> $a.gettype()

IsPublic IsSerial Name                                     BaseType
-------- -------- ----                                     --------
True     True     Hashtable                                System.Object
```

Member Retrival

Here unlike arrays index cannot be used to retrieve the members. The members are retrieved directly as shown (search time is very less)

$a['location'] - we get the output as Singapore

```
PS C:\Users\lenovo> $a['location']
Singapore
PS C:\Users\lenovo> $a['age']
40
PS C:\Users\lenovo>
```

To Get the methods and properties of Hash table

Here we use **Get-member -inputobject $a**

```
PS C:\Users\lenovo> get-member -inputobject $a


   TypeName: System.Collections.Hashtable

Name              MemberType            Definition
----              ----------            ----------
Add               Method                void Add(System.Object key, System.Object value), void IDictionary.Ad
Clear             Method                void Clear(), void IDictionary.Clear()
Clone             Method                System.Object Clone(), System.Object ICloneable.Clone()
Contains          Method                bool Contains(System.Object key), bool IDictionary.Contains(System.Ob
ContainsKey       Method                bool ContainsKey(System.Object key)
ContainsValue     Method                bool ContainsValue(System.Object value)
CopyTo            Method                void CopyTo(array array, int arrayIndex), void ICollection.CopyTo(arr
Equals            Method                bool Equals(System.Object obj)
GetEnumerator     Method                System.Collections.IDictionaryEnumerator GetEnumerator(), System.Coll
GetHashCode       Method                int GetHashCode()
GetObjectData     Method                void GetObjectData(System.Runtime.Serialization.SerializationInfo inf
GetType           Method                type GetType()
OnDeserialization Method                void OnDeserialization(System.Object sender), void IDeserializationCa
Remove            Method                void Remove(System.Object key), void IDictionary.Remove(System.Object
ToString          Method                string ToString()
Item              ParameterizedProperty System.Object Item(System.Object key) {get;set;}
Count             Property              int Count {get;}
IsFixedSize       Property              bool IsFixedSize {get;}
IsReadOnly        Property              bool IsReadOnly {get;}
IsSynchronized    Property              bool IsSynchronized {get;}
Keys              Property              System.Collections.ICollection Keys {get;}
SyncRoot          Property              System.Object SyncRoot {get;}
Values            Property              System.Collections.ICollection Values {get;}
```

Using Keys and Values

$a.keys and $a.values

```
PS C:\Users\lenovo> $a.keys
Name
Age
Location
PS C:\Users\lenovo> $a.values
John
40
Singapore
```

To change the Existing value In Hash table

$a.name = "james"
$a

```
PS C:\Users\lenovo> $a.name = "james"
PS C:\Users\lenovo> $a

Name                           Value
----                           -----
Name                           james
Age                            40
Location                       Singapore
```

To add Duplicate keys in Hash Table

$a.add("name" , "test")
We get an error
NOTE : in Hash Table Duplicate Keys Are not Allowed

```
PS C:\Users\lenovo> $a.add("name", "test")
Exception calling "Add" with "2" argument(s): "Item has already been added. Key in dictionary: 'Name'  Key b
added: 'name'"
At line:1 char:1
+ $a.add("name", "test")
+ ~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
    + FullyQualifiedErrorId : ArgumentException
```

Now we will add different key in hash table
$a.add("demo","test")
$a.demo

```
PS C:\Users\lenovo> $a.add("demo", "test")
PS C:\Users\lenovo> $a

Name                           Value
----                           -----
demo                           test
Name                           james
Age                            40
Location                       Singapore


PS C:\Users\lenovo> $a.demo
test
```

$a.containskey("test")

To check whether the keys and values exist

$a.containValue("test")
$a.containKey("demo")

```
PS C:\Users\lenovo> $a.containskey("demo")
True
PS C:\Users\lenovo> $a.containsValue("test")
True
PS C:\Users\lenovo>
```

GetEnumerator method

Here if we want output in table
$a.GetEnumerator()

```
PS C:\Users\lenovo> $a.getEnumerator()

Name                           Value
----                           -----
demo                           test
Name                           james
Age                            40
Location                       Singapore
```

$a.IsFixedSize - we get false as it is not fixed size

To Remove the value from the table

We use : Remove command - this command only operates at key
$.remove("demo") - make sure u put key name but not value name.
$a.getenumerator() - to check

To count the key-value pairs in the table

$a.count - the output we get is 3

```
PS C:\Users\lenovo> $a.count
3
```

To Create a Object using Hash Table

$b = [PSCustomObject]@{
Servicename = "test service"
Status = "running"
}
To check
$b.serviceName

```
PS C:\Users\lenovo> $b=[PSCustomObject]@{
>> Servicename="test service"
>> status = "running"
>> }
PS C:\Users\lenovo> $b.servicename
test service
PS C:\Users\lenovo>
```

To store the output of cmdlet in hash table

Here we even list the status of services running
$b =get-service | group-object status -AsHashTable -AsString

```
PS C:\Users\lenovo> $b =get-service | group-object  status -AsHashTable -AsString
>>
PS C:\Users\lenovo> $b

Name                           Value
----                           -----
Stopped                        {AarSvc_4bf2e02, AJRouter, ALG, AppIDSvc...}
Running                        {AdobeARMservice, Appinfo, AppXSvc, AudioEndpointBuilder...}
```

TO check the services running and stopped we use this

```
PS C:\Users\lenovo> $b.running | select-object -last 10

Status   Name               DisplayName
------   ----               -----------
Running  WdNisSvc           Microsoft Defender Antivirus Networ...
Running  WinDefend          Microsoft Defender Antivirus Service
Running  WinHttpAutoProx... WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt            Windows Management Instrumentation
Running  WlanSvc            WLAN AutoConfig
Running  wlidsvc            Microsoft Account Sign-in Assistant
Running  WpnService         Windows Push Notifications System S...
Running  WpnUserService_... Windows Push Notifications User Ser...
Running  wscsvc             Security Center
Running  WSearch            Windows Search


PS C:\Users\lenovo> $b.stopped | select-object -last 10

Status   Name               DisplayName
------   ----               -----------
Stopped  WMPNetworkSvc      Windows Media Player Network Sharin...
Stopped  workfolderssvc     Work Folders
Stopped  WpcMonSvc          Parental Controls
Stopped  WPDBusEnum         Portable Device Enumerator Service
Stopped  wuauserv           Windows Update
Stopped  WwanSvc            WWAN AutoConfig
Stopped  XblAuthManager     Xbox Live Auth Manager
Stopped  XblGameSave        Xbox Live Game Save
Stopped  XboxGipSvc         Xbox Accessory Management Service
Stopped  XboxNetApiSvc      Xbox Live Networking Service
```

To check the number of services running and stooped we use the command
$b.stopped .count
$b.running. count

```
PS C:\Users\lenovo> $b.stopped.count
166
PS C:\Users\lenovo> $b.running.count
111
PS C:\Users\lenovo>
```